

CSI 3140 - Laboratory 4 Report

Client-Server Interaction and Modern Web Services

Course Code	CSI 3140 - WWW Structures, Techniques and Standards
Laboratory	Laboratory 4 - Client-Server Interaction and Modern Web Services
Term	Spring/Summer 2026
Team Members	[Your Name] — [Student Number] [Teammate Name] — [Student Number] (if applicable)
Project Topic	Workshop and Event Registration System
Submission Date	July 11, 2026

1. Project Description

This project is a small client-server Web application for managing workshop and event registrations, one of the example topics suggested in the lab statement. Users can view a list of available workshops (title, category, date, capacity, and remaining seats), register for a seat in a workshop, and add new workshops through a form. The back end is a REST API built with Node.js and Express.js; the front end is a static HTML/CSS/JavaScript page that communicates with the API using `fetch()`.

2. File Structure

```
lab4-project/
├── public/
│   ├── index.html      Main client page
│   ├── styles.css      Client page styling
│   └── script.js        fetch() calls and DOM updates
├── data/
│   └── workshops.json   Seed data (5 workshops, loaded at server startup)
├── api-docs/
│   └── openapi.yaml     OpenAPI 3.0 description of the API
├── screenshots/         Testing and client screenshots
├── server.js            Express server and REST API routes
├── package.json         Node.js project configuration
├── README.md            Install / run / test instructions
├── testing-evidence.txt  Raw curl output for all endpoints
└── report.pdf           This report
```

3. Running the Project

1. Install dependencies: `npm install` (run once, inside `lab4-project/`)
2. Start the server: `npm start` (or `node server.js`)

3. Open a browser to `http://localhost:3000` — Express serves the front end from the `public/` folder, so no separate step is needed to "open" the client.

4. REST API Endpoints

The API exposes seven endpoints (more than the required four), all under the `/api` resource path, using nouns rather than verbs:

Method	Endpoint	Purpose	Success	Error
GET	<code>/api/status</code>	Health check for the API	200 OK	—
GET	<code>/api/workshops</code>	Return the list of all workshops	200 OK	—
GET	<code>/api/workshops/:id</code>	Return a single workshop	200 OK	404 Not Found
POST	<code>/api/workshops</code>	Create a new workshop	201 Created	400 Bad Request
PUT	<code>/api/workshops/:id</code>	Update an existing workshop	200 OK	400 / 404
PATCH	<code>/api/workshops/:id/register</code>	Reserve one seat in a workshop	200 OK	400 (full) / 404
DELETE	<code>/api/workshops/:id</code>	Remove a workshop	200 OK	404 Not Found

Full request/response formats, parameters, and examples are documented in `api-docs/openapi.yaml` (OpenAPI 3.0 format).

5. Example JSON Request and Response

POST `/api/workshops` (request)

```
{
  "title": "Intro to Docker",
  "category": "Workshop",
  "date": "2026-08-01",
  "capacity": 20
}
```

Response — 201 Created

```
{
  "id": 6,
  "title": "Intro to Docker",
  "category": "Workshop",
  "date": "2026-08-01",
  "capacity": 20,
  "seatsRemaining": 20
}
```

Response — 400 Bad Request (missing fields)

```
{
  "error": "Title, category, date, and capacity are required."
}
```

```
}
```

Response — 404 Not Found (GET /api/workshops/999)

```
{
  "error": "Workshop not found."
}
```

6. Front-End / Back-End Interaction

On page load, `script.js` calls `fetch("/api/workshops")` and renders the returned array as a list, with one "Register" button per workshop. Clicking Register sends a PATCH request to `/api/workshops/:id/register` and re-renders the list with the updated seat count (dynamic update). The "Add a New Workshop" form sends a POST request with the form data as a JSON body; the response is used to show either a green success message or a red validation error message under the form, without reloading the page.

7. Validation and Error Handling

- POST and PUT require title, category, date, and capacity; missing fields return **400 Bad Request**.
- capacity must be a positive number, checked on the server (not just the client).
- Requests for a workshop id that does not exist return **404 Not Found** with a JSON error message.
- Registering for a full workshop returns **400 Bad Request** ("This workshop is full.").
- An Express error-handling middleware catches unexpected exceptions and returns a generic **500** message instead of leaking stack traces to the client.

8. API Testing Evidence

All endpoints were tested with curl against the running server (see `testing-evidence.txt` for the full raw output) and manually through the browser. Summary of results:

Test	Result
GET /api/status	200 OK
GET /api/workshops	200 OK, 5 seed workshops returned
GET /api/workshops/1 (valid id)	200 OK
GET /api/workshops/999 (missing resource)	404 Not Found
POST /api/workshops (valid body)	201 Created
POST /api/workshops (missing fields, invalid)	400 Bad Request
PATCH /api/workshops/1/register	200 OK, seatsRemaining decremented
DELETE /api/workshops/999 (missing resource)	404 Not Found

Browser console and server terminal were both checked while testing; no errors were produced.

Screenshot 1: Front-end client page displaying data loaded from the API

Workshop and Event Registration System

CSI 3140 - Lab 4 - Client-Server Interaction and Modern Web Services

Available Workshops

REST API Fundamentals (Workshop) - 2026-07-15 - 11/30 seats left

Register

SOAP and WSDL Legacy Systems (Lecture) - 2026-07-17 - 40/40 seats left

Register

GraphQL Introduction (Seminar) - 2026-07-20 - 5/25 seats left

Register

Intro to gRPC and Protocol Buffers (Seminar) - 2026-07-22 - 20/20 seats left

Register

Building Client-Server Apps with Node.js (Workshop) - 2026-07-25 - 18/35 seats left

Register

Intro to Docker (Workshop) - 2026-08-01 - 20/20 seats left

Register

Add a New Workshop

Title

Category

Date



Capacity

Create Workshop

Screenshot 2: Successful POST request creating a workshop (dynamic success message)

Workshop and Event Registration System

CSI 3140 - Lab 4 - Client-Server Interaction and Modern Web Services

Available Workshops

REST API Fundamentals (Workshop) - 2026-07-15 - 11/30 seats left

Register

SOAP and WSDL Legacy Systems (Lecture) - 2026-07-17 - 40/40 seats left

Register

GraphQL Introduction (Seminar) - 2026-07-20 - 5/25 seats left

Register

Intro to gRPC and Protocol Buffers (Seminar) - 2026-07-22 - 20/20 seats left

Register

Building Client-Server Apps with Node.js (Workshop) - 2026-07-25 - 18/35 seats left

Register

Intro to Docker (Workshop) - 2026-08-01 - 20/20 seats left

Register

Test Screenshot Workshop (Workshop) - 2026-08-05 - 15/15 seats left

Register

Add a New Workshop

Title

Category

Date



Capacity



Create Workshop

Workshop "Test Screenshot Workshop" created successfully.

Screenshot 3: Invalid request showing a validation error message

Workshop and Event Registration System

CSI 3140 - Lab 4 - Client-Server Interaction and Modern Web Services

Available Workshops

REST API Fundamentals (Workshop) - 2026-07-15 - 11/30 seats left

Register

SOAP and WSDL Legacy Systems (Lecture) - 2026-07-17 - 40/40 seats left

Register

GraphQL Introduction (Seminar) - 2026-07-20 - 5/25 seats left

Register

Intro to gRPC and Protocol Buffers (Seminar) - 2026-07-22 - 20/20 seats left

Register

Building Client-Server Apps with Node.js (Workshop) - 2026-07-25 - 18/35 seats left

Register

Intro to Docker (Workshop) - 2026-08-01 - 20/20 seats left

Register

Test Screenshot Workshop (Workshop) - 2026-08-05 - 15/15 seats left

Register

Add a New Workshop

Title

Category

Date

 

Capacity

Create Workshop

Title, category, date, and capacity are required.

9. Legacy Web Services: SOAP, WSDL, XML Schema, and JAX-RPC

SOAP (Simple Object Access Protocol) is an XML-based messaging protocol used to exchange structured information between applications, typically over HTTP. Every SOAP message is an XML "envelope" containing a header and a body describing the operation being invoked and its parameters.

WSDL (Web Services Description Language) is an XML format that describes a SOAP service as a contract: what operations are available, what parameters they take, what they return, and where the service is hosted. Client tools use the WSDL to auto-generate code that can call the service.

XML Schema (XSD) defines the structure, data types, and constraints of the XML documents exchanged in SOAP messages, similar to how a REST API might use a JSON Schema to describe request/response shapes.

JAX-RPC (Java API for XML-based RPC) was a Java standard that let developers expose Java methods as SOAP Web services and generate client stubs from a WSDL file, making remote calls look like ordinary local method calls.

These technologies are considered legacy for most new Web development because they require heavier tooling (WSDL/XSD generation, SOAP envelopes), are verbose compared to JSON, and are harder to consume from browsers and mobile clients than REST + JSON. They remain common in **banking, government, healthcare (e.g., HL7-based systems), and other large enterprise systems** that were built years ago and still rely on stable, contract-based integrations.

SOAP/WSDL vs REST/OpenAPI Comparison

Aspect	SOAP/WSDL	REST/OpenAPI
Message format	XML	Usually JSON
Contract	WSDL	OpenAPI
Style	Operation-based	Resource-based
Complexity	Higher	Lower
Common use today	Enterprise and government systems	Public APIs and modern Web applications

10. Modern API Comparison: REST, GraphQL, and gRPC

API Style	Main Format	Best Use Case	Main Strength
REST + OpenAPI	JSON over HTTP	Public and CRUD-style APIs	Simple and widely supported
GraphQL	JSON over HTTP	Client-driven data retrieval	Flexible queries
gRPC	Protocol Buffers over HTTP/2	Internal microservices	Fast and strongly typed

REST is a good default for most Web APIs because it maps naturally onto resources and standard HTTP verbs/status codes, is easy to test with common tools (curl, Postman, the browser), and is universally supported by front-end frameworks. **GraphQL** becomes useful when different clients need different shapes or amounts of data from the same backend and want to avoid over-fetching or under-fetching (e.g., a mobile app and a web dashboard both querying a shared schema but requesting different fields). **gRPC** is most useful for internal, high-performance communication between microservices, where both ends are known in advance and can share a strongly-typed Protocol Buffers contract.

For the Lab 4 Workshop and Event Registration System, **REST** was the most appropriate choice: the application is a simple public-facing CRUD system (list, view, create, update, register, delete workshops), it needs to be consumed directly by a browser using `fetch()`, and the project explicitly requires a manually implemented Express-based REST API rather than GraphQL or gRPC tooling.

11. Basic Security and Error Handling Discussion

The API validates required fields and value types on the server (not only in the browser), returns generic, safe error messages instead of raw stack traces, and uses appropriate HTTP status codes (400 for bad input, 404 for missing resources, 500 for unexpected server errors). In a real production deployment, additional security would be required, including:

- **Authentication and authorization** — verifying who is calling the API and what they are allowed to do (e.g., only admins can delete workshops).
- **HTTPS** — encrypting traffic between the browser and server so data and credentials cannot be intercepted.
- **Rate limiting** — preventing abuse or denial-of-service by limiting how many requests a client can make in a given period.
- **CORS configuration** — restricting which origins are allowed to call the API in a browser context.
- **Stricter input validation and sanitization** — guarding against malformed or malicious payloads beyond the basic checks implemented here.

12. Deployment Thinking

The API could be deployed to a Node.js-friendly host (e.g., Render, Railway, an Azure/AWS VM, or a container platform). A real deployment would require: setting `PORT` from an environment variable, replacing the in-memory array with a persistent database, adding HTTPS/TLS termination, configuring CORS for the real front-end domain, and adding logging/monitoring for production traffic.

13. Checklist

Item	Completed?
The project includes a clear client-server structure.	Yes
The server runs using Node.js and Express.js.	Yes
The server provides at least four REST API endpoints.	Yes (7)
The API uses JSON request and response bodies.	Yes
The API uses appropriate HTTP methods.	Yes
The API uses appropriate HTTP status codes.	Yes
The API includes server-side validation.	Yes
The API returns clear error messages for invalid requests.	Yes
The client uses <code>fetch()</code> to call the API.	Yes
The client dynamically displays data received from the API.	Yes
The client includes a form that sends data to the API.	Yes
API testing screenshots are included.	Yes
The report includes API documentation or an OpenAPI-style description.	Yes
The report explains SOAP, WSDL, XML Schema, and JAX-RPC.	Yes
The report compares SOAP/WSDL with REST/OpenAPI.	Yes
The report compares REST, GraphQL, and gRPC.	Yes
The project was tested in a browser.	Yes

The browser console and server terminal were checked for errors.	Yes
A README file explains how to run the project.	Yes

14. Reflection

[Fill in your own reflection before submitting — e.g., any issues you personally ran into while setting up Node/Express, debugging a validation rule, or getting `fetch()` to update the page, and how you resolved them. The TA can ask any team member to explain this during the demonstration, so make sure it reflects what you actually did.]